Universidad Católica Boliviana “San Pablo”
Facultad de Ingeniería
Departamento de Ingeniería de Sistemas

La Paz - Bolivia

GRUPO 5 S.R.L

Sistema POS para Cafetería

Integrantes:

- Claros Suntura Juan Jose *(Desarrollador FullStack)*
- Lecoña Condori Elias Milan *(Desarrollador FullStack)*
- Maldonado Carvajal Alan Ariel *(Scrum Master)*
- Torrez Calle Alvaro Ariel *(Product Owner)*

Materia: Ingenieria de Software (SIS-213)
Docente: Miguel Angel Pacheco Arteaga
Fecha: 06/03/2026
Universidad: Universidad Catolica Boliviana

Índice General

Capítulo 1. MARCO REFERENCIAL DEL SISTEMA TPS **1**

1.1 Introducción 1

1.2 Antecedentes 1

1.2.1 Antecedentes del objeto de estudio 1

1.2.2 Referencias técnicas de otros sistemas TPS 2

1.3 Descripción del objeto de estudio 2

1.4 Identificación del Problema 3

1.5 Formulación del Problema 3

1.6 Objetivos 3

1.6.1 Objetivo General 3

1.6.2 Objetivos Específicos 3

1.7 Justificación 4

1.7.1 Justificación Técnica 4

1.7.2 Justificación Organizacional 4

1.7.3 Justificación Económica 4

1.8 Límites y Alcances 5

1.8.1 Límites 5

1.8.2 Alcances 5

1.9 Metodología del Proyecto 5

1.9.1 Tipo de estudio 5

1.9.2 Metodología de desarrollo 6

1.9.3 Técnicas 6

1.10 Análisis preliminar del sistema TPS 6

1.11 Propuesta de solución 6

1.12 Cronograma 7

Capítulo 2. MARCO TEÓRICO DEL SISTEMA TPS **8**

2.1 Sistemas de Información Organizacional 8

2.1.1 Definición 8
2.1.2 Componentes y Tipos 8
2.2 Sistema de Procesamiento de Transacciones (TPS) 8
2.3 Arquitectura de sistemas web 9
2.4 Seguridad en sistemas de información 9
2.5 Base de datos 9
2.6 Metodología de desarrollo 10

Capítulo 3. MARCO PRÁCTICO — DESARROLLO DEL SISTEMA TPS 11

3.1 Análisis del sistema 11
3.2 Determinación de requerimientos 12
3.2.1 Requerimientos funcionales 12
3.2.2 Requerimientos no funcionales 12
3.3 Modelado del sistema 13
3.3.1 Historias de Usuario 13
3.3.2 Diagramas UML 13
3.4 Diseño del sistema 14
3.4.1 Arquitectura del sistema 14
3.5 Diseño de la Base de Datos 14
3.6 IMPLEMENTACIÓN DE LOS MÓDULOS DEL SISTEMA 15
3.6.1 Módulo de Gestión de Procesos 15
3.6.2 Módulo de Usuarios y Roles 15
3.6.3 Módulo de Transacciones 15
3.6.4 Módulo de Reportes 15
3.7 Capa Backend Funcional 16
3.8 Validación y pruebas del sistema 16
3.9 Desarrollo del prototipo funcional 17
3.10 Documentación de Ingeniería Completa 17

Capítulo 4. CONCLUSIONES Y RECOMENDACIONES 18

4.1 Conclusiones 18

4.2 Recomendaciones 18

Índice de Figuras, Tablas y Diagramas

Figuras

1 Ejemplo de prueba en el análisis del sistema 11

Tablas

1 Cronograma sugerido de desarrollo del proyecto 7
2 Ejemplo de diccionario para tabla de Base de Datos 14

Diagramas

1 Ejemplo general de Diagrama Estructural UML 13

Capítulo 1. MARCO REFERENCIAL DEL SISTEMA TPS

1.1 Introducción

Los Sistemas de Información Organizacional cumplen un rol fundamental en la actualidad, permitiendo a las empresas gestionar sus operaciones de manera eficiente y centralizada. En este contexto, la evolución de los sistemas de Procesamiento de Transacciones (TPS, por sus siglas en inglés) ha sido clave para la automatización de procesos operativos rutinaros, garantizando la consistencia y seguridad de la información.

La necesidad de automatización de procesos se hace evidente ante el crecimiento de las transacciones diarias en cualquier organización moderna, donde el registro manual desencadena errores humanos, pérdida de tiempo y falta de trazabilidad. El uso de sistemas web organizacionales centraliza esta gestión, permitiendo el acceso concurrente desde diferentes ubicaciones y dispositivos de forma segura.

Ejemplo de enfoque: Este proyecto consiste en el desarrollo de un Sistema de Información Organizacional Web basado en la evolución del enfoque TPS, que permitirá gestionar procesos, usuarios, transacciones y reportes mediante una plataforma tecnológica accesible y segura.

1.2 Antecedentes

1.2.1 Antecedentes del objeto de estudio

Actualmente, la organización lleva a cabo sus procesos de manera manual o semiautomatizada utilizando hojas de cálculo dispersas. Esta práctica genera inconsistencia en los datos y cuellos de botella en la operación diaria.

Ejemplos de problemas existentes:

- Registro manual de información sin validaciones de integridad.
- Falta de control granular de usuarios y permisos (todos acceden a toda la información).

- Dificultad para generar reportes consolidados en tiempo real.
- Baja trazabilidad de transacciones (no se sabe qué usuario modificó un registro ni cuándo).

1.2.2 Referencias técnicas de otros sistemas TPS

Para el desarrollo de este proyecto se han analizado diversos sistemas similares en el mercado:

1. **Sistema Odoo (Módulos Base)**
   - **Tipo de sistema:** ERP / TPS escalable.
   - **Funcionalidades principales:** Gestión de ventas, inventario, usuarios y contabilidad transaccional.
   - **Tecnologías:** Construido en Python (*Backend*) y PostgreSQL.
   - **Diferencia con el sistema propuesto:** Odoo es una solución genérica y compleja, mientras que el sistema propuesto estará diseñado a la medida de los procesos específicos de la organización.
2. **Dolibarr ERP/CRM**
   - **Tipo de sistema:** Sistema TPS web modular para PYMES.
   - **Funcionalidades principales:** Control de usuarios, gestión de órdenes, facturación y operaciones comerciales.
   - **Tecnologías:** PHP y MySQL/MariaDB.
   - **Diferencia con el sistema propuesto:** Nuestro sistema ofrecerá una capa *frontend* moderna e interactiva (*Single Page Application*) enfocada en la rapidez de las transacciones.

1.3 Descripción del objeto de estudio

El proyecto se enmarca dentro de una organización (o escenario simulado) que requiere controlar el flujo diario de sus operaciones. Los procesos organizacionales principales incluyen el registro de entradas, asignación de tareas y validación continua.

- **Actores del sistema:** Administradores, Operadores y Auditores.
- **Flujo general del negocio:** Un operador registra una transacción inicial, la cual sigue un ciclo de vida de validación y eventual aprobación por parte del administrador.

1.4 Identificación del Problema

La organización presenta dificultades en la gestión de procesos, control de usuarios, registro de transacciones y generación de reportes debido a la ausencia de un sistema de información automatizado y centralizado, lo que incrementa los tiempos de respuesta y los márgenes de error operativo.

1.5 Formulación del Problema

¿Cómo desarrollar un Sistema de Información Organizacional Web basado en el enfoque TPS que permita gestionar procesos, usuarios, transacciones y reportes de manera eficiente y segura?

1.6 Objetivos

1.6.1 Objetivo General

Desarrollar un Sistema de Información Organizacional Web basado en el enfoque de Procesamiento de Transacciones (TPS) que permita gestionar procesos, usuarios, transacciones y reportes mediante una arquitectura *backend* funcional.

1.6.2 Objetivos Específicos

- Analizar los procesos organizacionales actuales y el flujo de información.
- Identificar los requerimientos funcionales y no funcionales del sistema.
- Diseñar la arquitectura del sistema web adoptando un modelo cliente-servidor.
- Implementar el módulo de usuarios y roles para el control de acceso.

- Implementar el módulo de gestión de procesos operativos.
- Implementar el módulo de transacciones garantizando la integridad de datos.
- Implementar el módulo de reportes y auditoría transaccional.
- Desarrollar la capa *backend* funcional consumible a través de una API REST.
- Validar el sistema mediante pruebas de *software* estructuradas.
- Documentar integralmente el sistema de *software*.

1.7 Justificación

1.7.1 Justificación Técnica

El uso de una arquitectura web (cliente-servidor a través de API REST) permite una alta escalabilidad y fácil mantenimiento. Se cuenta con un *backend* funcional robusto conectado a una base de datos relacional que asegura la integridad referencial. A nivel de seguridad, se incorporan esquemas de autenticación por *tokens* (fichas o vales digitales) y cifrado de credenciales.

1.7.2 Justificación Organizacional

La implementación directa mejorará la gestión de procesos al estandarizar los flujos de trabajo. Se establecerá un estricto control de usuarios, la automatización de transacciones reducirá la carga operativa, y la generación de reportes automáticos brindará soporte oportuno a la toma de decisiones.

1.7.3 Justificación Económica

El sistema promoverá la reducción de errores humanos, lo que se traduce en optimización de tiempo laboral. En el mediano plazo, la automatización reflejará una disminución de costos operativos asociados a retrabajos, uso de papel y pérdida de información.

1.8 Límites y Alcances

1.8.1 Límites

El sistema:

- Será exclusivamente web (no incluirá aplicaciones móviles nativas).
- Usará una base de datos relacional para garantizar transacciones consistentes.
- Tendrá un sistema cerrado de autenticación de usuarios (no integrará inicios de sesión sociales).
- No incluirá integración directa con sistemas de *software* de terceros o pasarelas de pago.

1.8.2 Alcances

El sistema permitirá:

- Gestionar y parametrizar procesos del giro del negocio.
- Gestionar usuarios, asignar roles y administrar permisos granulares.
- Registrar, editar, visualizar y anular transacciones de forma segura.
- Generar reportes tabulares y gráficos sobre la operatividad del sistema.
- Administrar información base y catálogos de apoyo.

1.9 Metodología del Proyecto

1.9.1 Tipo de estudio

Por su naturaleza, la investigación será de tipo **Aplicado** (orientado a la solución de un problema concreto), **Tecnológico** (mediante el uso de herramientas de *software*) y **Descriptivo** (para modelar el escenario y comportamiento actual).

1.9.2 Metodología de desarrollo

Se aplicará la metodología ágil ***Scrum***, que se adapta idealmente al desarrollo de *software* iterativo e incremental:

- ***Sprints*** (Iteraciones): Ciclos de trabajo de 2 a 4 semanas.
- ***Product Backlog*** (Pila de producto): Lista priorizada de todos los requerimientos y módulos del sistema.
- ***Sprint Backlog*** (Pila del ciclo): Tareas seleccionadas para ser desarrolladas en el ***Sprint*** actual.
- **Entregables:** Prototipos funcionales al final de cada iteración demostrando valor.

1.9.3 Técnicas

- **Entrevistas:** Al personal clave de la organización para levantar requerimientos.
- **Observación directa:** De los procesos manuales actuales en el sitio.
- **Análisis documental:** Revisión de formatos, reportes o planillas actuales.
- **Modelado UML:** Para representar gráficamente el diseño de la solución (casos de uso, diagramas de clase, etc.).

1.10 Análisis preliminar del sistema TPS

La organización actual cuenta con un déficit de integración. Los procesos operativos dependen en gran medida del papel, lo que retrasa la respuesta al usuario final. La información crítica se almacena en computadoras aisladas, siendo altamente vulnerable. Las transacciones no mantienen un historial de auditoría y los usuarios no operan bajo un modelo de privilegios.

1.11 Propuesta de solución

- **Sistema:** Sistema de Información Organizacional Web enfocado en TPS.
- **Arquitectura:** Patrón Cliente — Servidor con separación de responsabilidades (*Frontend* y *Backend*).

Tecnologías sugeridas:

- ***Backend*:** Node.js, Spring Boot, o Django.
- **Base de datos:** Motor relacional como PostgreSQL o MySQL.
- ***Frontend*** (Interfaz): Librería React, Angular, o una estructura base en HTML, CSS y JavaScript.

1.12 Cronograma

El despliegue del proyecto abarca un tiempo estimado de desarrollo según las siguientes fases:

Fases	Entregables / Hitos	Duración
Análisis	Captura de requerimientos, diseño UI/UX y diagrama de base de datos.	2 Semanas
Diseño	Arquitectura tecnológica, modelado UML y definición técnica.	2 Semanas
Desarrollo	Programación *backend*, *frontend* y construcción de la base de datos.	6 Semanas
Pruebas	Ejecución de validaciones, corrección de defectos lógicos (*bugs*).	2 Semanas
Documentación	Finalización de manuales y armado del documento de grado.	2 Semanas

Tabla 1: Cronograma sugerido de desarrollo del proyecto

Capítulo 2. MARCO TEÓRICO DEL SISTEMA TPS

2.1 Sistemas de Información Organizacional

2.1.1 Definición

Un sistema de información organizacional es un conjunto de componentes interrelacionados que recolectan, procesan, almacenan y distribuyen información para apoyar la toma de decisiones, el control y la coordinación dentro de una entidad.

2.1.2 Componentes y Tipos

Involucran equipo (*hardware*), programas (*software*), bases de datos, redes, procedimientos y recursos humanos. Existen diferentes tipos, como los Sistemas de Soporte a Decisiones (DSS), de Información Gerencial (MIS) y los Sistemas de Procesamiento de Transacciones (TPS).

2.2 Sistema de Procesamiento de Transacciones (TPS)

El TPS es la columna vertebral de cualquier sistema organizacional que recolecta y procesa las transacciones generadas en el día a día.

- **Características principales:** Procesamiento rápido, alta confiabilidad, capacidad para manejar gran volumen de datos de forma estandarizada y estricta integridad en cada transacción.
- **Funciones:** Captura de datos de origen, verificación, procesamiento inmediato, actualización de bases de datos maestras y emisión de acuses de recibo.
- **Evolución hacia sistemas web:** Han pasado de terminales cerradas monolíticas a integrarse mediante la web, lo cual otorga ubicuidad y permite operaciones distribuidas.

2.3 Arquitectura de sistemas web

- **Cliente:** La interfaz que interactúa con el usuario final a través de un navegador web, responsable de la presentación de la información.
- **Servidor (*Backend*):** La computadora central o contenedor de la nube que procesa la lógica de negocios, gestionando la concurrencia y procesando los datos.
- **API (*Application Programming Interface*):** El puente de comunicación documentado entre el Cliente y el Servidor.
- **Base de datos:** Repositorio en el que se almacenan de manera persistente las entidades transaccionales.

2.4 Seguridad en sistemas de información

- **Autenticación:** El proceso de validar la identidad del usuario (ej. usuario y contraseña).
- **Autorización:** Determinación de los recursos a los cuales un usuario autenticado tiene permitido acceder.
- **Roles:** Agrupaciones lógicas de permisos y privilegios que pueden ser asignados masivamente.
- **Control de acceso:** Políticas implementadas a nivel de *backend* y *frontend* para restringir el acceso indebido a información crítica.

2.5 Base de datos

- **Modelo relacional:** Enfoque donde la información se organiza en tablas compuestas por filas y columnas vinculadas entre sí rigurosamente.
- **Integridad:** Reglas implementadas para mantener consistencia, exactitud y validez de la información frente a inserciones u operaciones anómalas.
- **Normalización:** Proceso de diseño estructural que elimina la redundancia de datos.

- **Transacciones:** Secuencias de operaciones de base de datos que se tratan como una unidad indivisible (si una operación falla, todo se revierte).

2.6 Metodología de desarrollo

Scrum Es un marco de trabajo colaborativo que permite abordar proyectos complejos.

- **Roles:** ***Product Owner*** (Dueño del producto), ***Scrum Master*** (Facilitador) y ***Developers*** (Equipo de desarrollo).
- **Artefactos:** ***Product Backlog*** (Pila de producto), ***Sprint Backlog*** (Pila del ciclo) e Incremento del Producto.
- **Eventos:** ***Sprint Planning*** (Planificación), ***Daily Scrum*** (Reunión diaria), ***Sprint Review*** (Revisión) y ***Sprint Retrospective*** (Retrospectiva).

Capítulo 3. MARCO PRÁCTICO — DESARROLLO DEL SISTEMA TPS

3.1 Análisis del sistema

En base a la recopilación de datos, la estructura organizacional del sistema identifica diversos actores: Administradores, quienes tienen control global; y Operadores, quienes ejecutan diariamente transacciones asociadas a los procesos de negocio. El flujo de procesos demanda digitalizar desde el ingreso del actor al sistema, registro de eventos, hasta la confirmación e impacto histórico.

Figura 1: Ejemplo de prueba en el análisis del sistema

3.2 Determinación de requerimientos

3.2.1 Requerimientos funcionales

Los requerimientos funcionales expresan lo que el sistema **debe hacer** operativamente.

- **RF01:** El sistema permitirá registrar nuevos usuarios operativos.
- **RF02:** El sistema permitirá a los usuarios iniciar y cerrar sesión de manera segura.
- **RF03:** El sistema permitirá al administrador asignar o revocar roles de acceso.
- **RF04:** El sistema permitirá registrar un nuevo proceso organizacional básico.
- **RF05:** El sistema permitirá el registro consecutivo de transacciones asociadas a un proceso.
- **RF06:** El sistema permitirá generar reportes tabulados basados en un rango de fechas.

3.2.2 Requerimientos no funcionales

Establecen las restricciones y la forma en cómo debe operar y comportarse estructuralmente la aplicación.

- **Seguridad:** Encriptación de contraseñas usando algoritmos seguros (ej. *bcrypt*) y comunicaciones seguras.
- **Rendimiento:** Tiempos de respuesta para guardado de transacciones menores a 2 segundos en carga moderada.
- **Usabilidad:** Interfaces intuitivas, adaptables a monitores de escritorio (diseño responsivo).
- **Disponibilidad:** Arquitectura preparada para mantener disponibilidad en un entorno de servidor las 24 horas.
- **Escalabilidad:** Separación modular del código que facilite agregar futuros módulos sin modificar el núcleo operativo.

3.3 Modelado del sistema

3.3.1 Historias de Usuario

Se definen detallando la necesidad y la regla de aceptación:

- *Como* administrador, *quiero* registrar y eliminar usuarios *para* controlar estrictamente el acceso a la plataforma corporativa.
- *Como* operador de sistemas, *quiero* registrar una transacción en tiempo de ejecución *para* avanzar en mi cuota diaria de procesos.

3.3.2 Diagramas UML

A continuación se muestra un ejemplo genérico de la estructura de un diagrama, en este caso, se deben incrustar aquí los diagramas correspondientes: Diagramas de Casos de Uso, Clases, Secuencia y Actividades generados en herramientas tipo StarUML o Drawio.

Diagrama 1: Ejemplo general de Diagrama Estructural UML

3.4 Diseño del sistema

3.4.1 Arquitectura del sistema

Modelo: Arquitectura Web Cliente-Servidor El sistema se dividirá lógicamente en una aplicación cliente basada en componentes interactuando asíncronamente con un servidor *backend* que expondrá puntos de enlace (*endpoints*) REST. *Se sugiere aplicar adicionalmente el modelo C4 para diagramar las capas de contexto, contenedores, y componentes de la solución.*

3.5 Diseño de la Base de Datos

Se diseña bajo el paradigma relacional para modelar las entidades y su cardinalidad. Las restricciones aseguran que una transacción no puede existir sin su usuario operador o su proceso maestro.

Campo	Tipo	Llave	Descripción
id	Int	Primaria (PK)	Identificador único de la transacción.
usuario_id	Int	Foránea (FK)	Código del usuario responsable.
fecha	Datetime	Ninguna	Fechas y hora de ejecución.
estado	Varchar	Ninguna	Estado lógico de la transacción.

Tabla 2: Ejemplo de diccionario para tabla de Base de Datos

3.6 IMPLEMENTACIÓN DE LOS MÓDULOS DEL SISTEMA

3.6.1 Módulo de Gestión de Procesos

Modulo fundamental para configurar el entorno.

- **Registro de procesos:** Interfaces para ingresar la descripción de un proceso.
- **Actualización:** Se permite modificar detalles exceptuando integridades históricas.
- **Eliminación y consulta:** Eliminación lógica de procesos inactivos y un panel de cuadrícula (*Grid*) para buscar por criterios múltiples.

3.6.2 Módulo de Usuarios y Roles

La barrera de seguridad del sistema TPS.

- **Registro e Inicio de sesión:** Validaciones fuertes contra la tabla criptográfica.
- **Gestión de roles y control de acceso:** El *frontend* renderizará menús diferenciados basados en el rol (Administrador versus Operativo) dictado por el *token* JWT emitido por la API *backend*.

3.6.3 Módulo de Transacciones

Núcleo central del objeto TPS, diseñado para alta velocidad operativa y baja fricción en la entrada de datos (*Data Entry*).

- **Registro:** Pantallas con autocompletado y validaciones estrictas.
- **Modificación/Consulta:** Vista detallada tipo “maestro-detalle” y bloqueo de interferencias concurrentes en caso de ediciones simultáneas.
- **Historial de operaciones:** Bitácora interna de acciones (“El usuario X anuló la transacción a las 15:42 p.m.”).

3.6.4 Módulo de Reportes

Módulo analítico que destila la información transaccional operativa.

- Generación en memoria del “Reporte estadístico mensual”.
- Extracción y consolidación de “Reporte de transacciones por usuario”, posibilitando descargas en formatos limpios o impresiones en formato PDF.

3.7 Capa Backend Funcional

El *backend* es responsable único del procesamiento transaccional aislado de la interfaz gráfica, diseñado bajo principios REST y patrones en capas:

- **Controladores (*Controllers*):** Capturan las peticiones HTTP y manejan la repuesta.
- **Servicios (*Services*):** Contienen puramente la lógica de reglas de negocio organizacionales.
- **Modelos/Entidades (*Models/Entities*):** Representación orientada a objetos de las tablas y procedimientos almacenados.
- **Capa de Seguridad (*Middlewares*):** Componentes intermedios que verifican autenticidad mediante la inspección de cabeceras HTTP antes de conceder cualquier ejecución.
- **Validaciones (*DTOs*):** Objetos de transferencia de datos (*Data Transfer Objects*) que verifican limpiamente los cuerpos de datos (*Payloads*) antes de impactar el Servicio.

3.8 Validación y pruebas del sistema

El sistema asegura la calidad del producto final a través de distintas evaluaciones de estrés y rendimiento:

- **Pruebas unitarias:** Validando que las funciones matemáticas y servicios individuales del *backend* operen según la lógica.
- **Pruebas funcionales:** Ejecución de casos de uso (Ej: Qué sucede si el usuario ingresa un formato de fecha erróneo).
- **Pruebas de integración:** Ensayos del flujo Cliente hacia la API y hacia la base

de datos extremo a extremo.

- **Pruebas de aceptación:** Pruebas finales realizadas con un entorno cercano a la organización para validación definitiva del *Product Owner*.

3.9 Desarrollo del prototipo funcional

A lo largo de los *sprints* se generan prototipos incrementales. En esta etapa el proyecto expone sus interfaces plenamente interactivas reflejando casos de éxito desde el inicio de sesión (*login*), hasta el registro exitoso de la operación. *(Incluir evidencias y capturas de pantalla reales aquí)*.

3.10 Documentación de Ingeniería Completa

Se acompañan como anexos técnicos o repositorios vinculados:

- **Documentación funcional:** Incluye la Especificación de Requerimientos de Software (SRS), relevamiento documentado explícito e historias de usuario extendidas.
- **Documentación técnica:** El diagrama de la arquitectura desplegada, diccionarios de datos, modelo E/R completo y especificación paramétrica de API.
- **Documentación del sistema:** Manual de usuario para operadores, el manual técnico, directrices de instalación en entorno de servidor y parametrización de variables de entorno.
- **Documentación del código:** Documentación generada automáticamente, estructura arquitectónica base (*Framework*), y lista de librerías vinculadas (*dependencias*).

Capítulo 4. CONCLUSIONES Y RECOMENDACIONES

4.1 Conclusiones

- Tras el proceso de desarrollo, se implementó satisfactoriamente el Sistema de Información Organizacional Web basado en el enfoque TPS para la organización, abordando exitosamente el problema de registro e integridad de datos operacionales.
- Los módulos principales (Usuarios, Procesos, Transacciones y Reportes) fueron construidos integralmente bajo la arquitectura cliente-servidor e implementados desplegando una capa *backend* robusta y funcional.
- Se cumplió a cabalidad el objetivo de centralización de la información, permitiendo a los actores automatizar su flujo de trabajo manual y erradicar asimetrías de información, fortaleciendo fundamentalmente la toma de decisiones empresariales.

4.2 Recomendaciones

- **Mejoras futuras y Escalabilidad:** Es recomendable planificar la ampliación modular incorporando herramientas de Inteligencia de Negocios (*Business Intelligence* - BI) para tableros de control avanzados.
- **Seguridad:** Monitorear activamente los registros de acceso al servidor y adoptar periódicamente actualizaciones en las librerías base (dependencias) para prevenir vulnerabilidades.
- **Integración con otros sistemas:** Se abre la puerta de escalar este sistema TPS transaccional a un formato ERP (*Enterprise Resource Planning*) ligero, integrando módulos adicionales de contabilidad o a futuro integrar servicios en la nube a través de peticiones extenas.